# ASSIGNMENT #6

jQuery: Selectors, Effects and Animations

Task 1 — Selector Detective • Task 2 — The Party in Motion

**Student:** [WRITE YOUR NAME] **Group:** [WRITE YOUR GROUP]

**GitHub repository:** [PASTE YOUR REAL GITHUB REPOSITORY URL]

**Report preparation note: Screenshots below were captured from a running browser test of this implementation. Verify them on your own computer and rewrite explanations in your own words before submission.**

## 1. Objective and implementation

The page recreates a birthday party using a JavaScript array for the 21 initial guests and four rooms on two floors. jQuery generates all guest cards from the model and stores clothing, student status, arrival time and belongings in classes and data-* attributes. Every case searches the DOM; results are highlighted and animated. Task 2 adds spotlight fading, animated guest details, guest movement and animated sorting.

## 2. Initial arrangement

| Room | Floor | Guests (left to right) |
|---|---|---|
| **Balcony** | 2 | Aigerim · Aliya · Madina · Ivan |
| **Hall** | 2 | Arman · Sophia · Dias · Bekzat · Aruzhan · Saule |
| **Kitchen** | 1 | Nurlan · Emma · Timur · Sergey · Yerlan · Kamila |
| **Living room** | 1 | Daniel · Zarina · Natasha · Askar · Inkar |

**Reference: Assignment PDF, page 2 (party illustration).**

## 3. Task 1 — Twenty DOM queries

### Case 01 — Who is in the hall?

```
$(`.room[data-room="hall"] .guest`)
```

**Expected output:** Arman, Sophia, Dias, Bekzat, Aruzhan, Saule

Explanation: The jQuery chain filters the page elements using DOM attributes, classes, position or related guests; it does not search the source array.

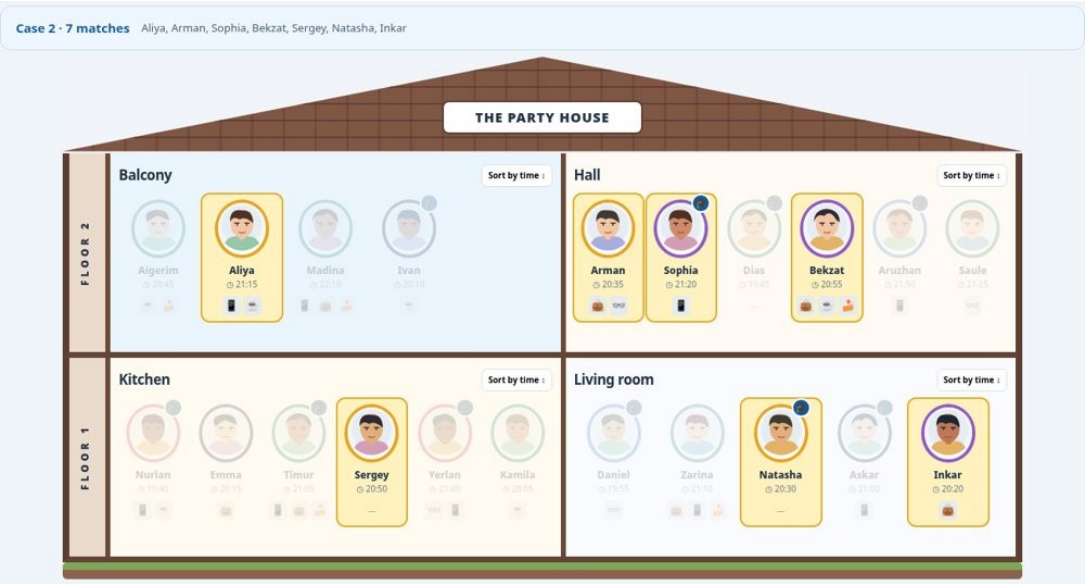



### Case 02 — Yellow or purple clothes?

$(`.guest.color-yellow, .guest.color-purple`)

**Expected output:** Aliya, Arman, Sophia, Bekzat, Sergey, Natasha, Inkar

Explanation: The jQuery chain filters the page elements using DOM attributes, classes, position or related guests; it does not search the source array.




### Case 03 — AITU students upstairs?

$(`.floor[data-floor="2"] .guest.aitu`)

**Expected output:** Ivan, Sophia, Dias, Aruzhan

Explanation: The jQuery chain filters the page elements using DOM attributes, classes, position or related guests; it does not search the source array.




### Case 04 — Glasses or a bag?

$(`.guest.has-glasses, .guest.has-bag`)

**Expected output:** Madina, Arman, Bekzat, Saule, Emma, Timur, Yerlan, Daniel, Zarina, Inkar

Explanation: The jQuery chain filters the page elements using DOM attributes, classes, position or related guests; it does not search the source array.




### Case 05 — Nothing at all?

$(`.guest:not([data-things~="phone"]):not([data-things~="glasses"]):not([data-things~="bag"]):not([data-things~="cup"]):not([data-things~="crumbs"])`)

**Expected output:** Dias, Sergey, Natasha

Explanation: The jQuery chain filters the page elements using DOM attributes, classes, position or related guests; it does not search the source array.

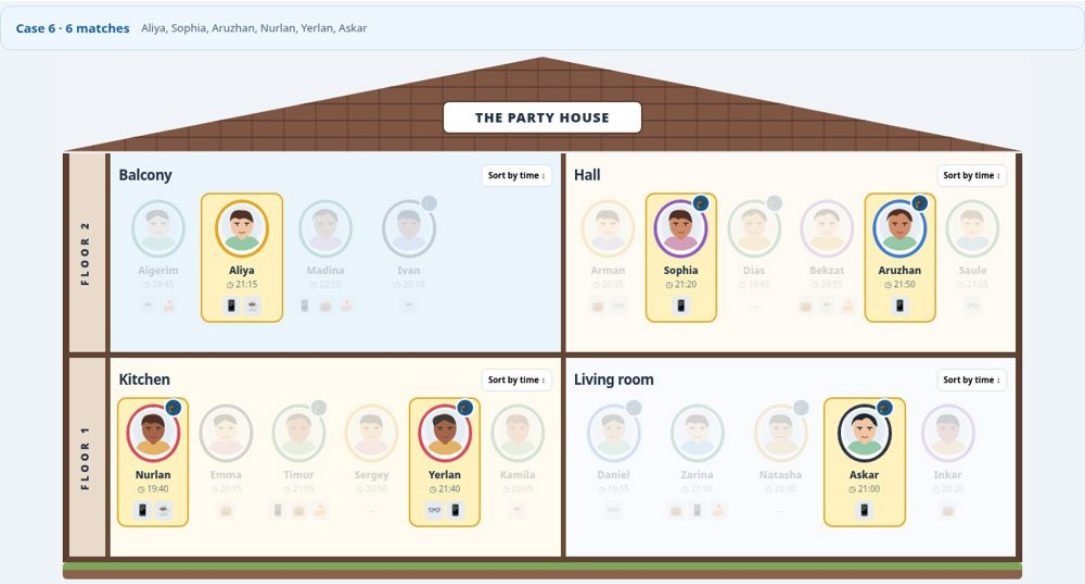



### Case 06 — Phone without bag?

$(`.guest.has-phone:not(.has-bag)`)

**Expected output:** Aliya, Sophia, Aruzhan, Nurlan, Yerlan, Askar

Explanation: The jQuery chain filters the page elements using DOM attributes, classes, position or related guests; it does not search the source array.




### Case 07 — 20:30–21:00 inclusive?

$(`.guest`).filter(function(){let t=+$(this).attr("data-minutes");return t>=1230&&t<=1260;})

**Expected output:** Aigerim, Arman, Bekzat, Sergey, Natasha, Askar

Explanation: The jQuery chain filters the page elements using DOM attributes, classes, position or related guests; it does not search the source array.




### Case 08 — After 21:00, not AITU?

```
$(`.guest.non-aitu`).filter(function(){return +$(this).attr("data-minutes")>1260;})
```

**Expected output:** Aliya, Madina, Saule, Zarina

Explanation: The jQuery chain filters the page elements using DOM attributes, classes, position or related guests; it does not search the source array.




### Case 09 — Next to Askar?

```
$(`.guest[data-name="Askar"]`).prev(".guest").add($(`.guest[data-name="Askar"]`).next(".guest"))
```

**Expected output:** Natasha, Inkar

Explanation: The jQuery chain filters the page elements using DOM attributes, classes, position or related guests; it does not search the source array.




### Case 10 — Last person in each room?

$(`.room .guest:last-child`)

**Expected output:** Ivan, Saule, Kamila, Inkar

Explanation: The jQuery chain filters the page elements using DOM attributes, classes, position or related guests; it does not search the source array.




### Case 11 — Third person in each room?

$(`.room .guest:nth-child(3)`)

**Expected output:** Madina, Dias, Timur, Natasha

Explanation: The jQuery chain filters the page elements using DOM attributes, classes, position or related guests; it does not search the source array.

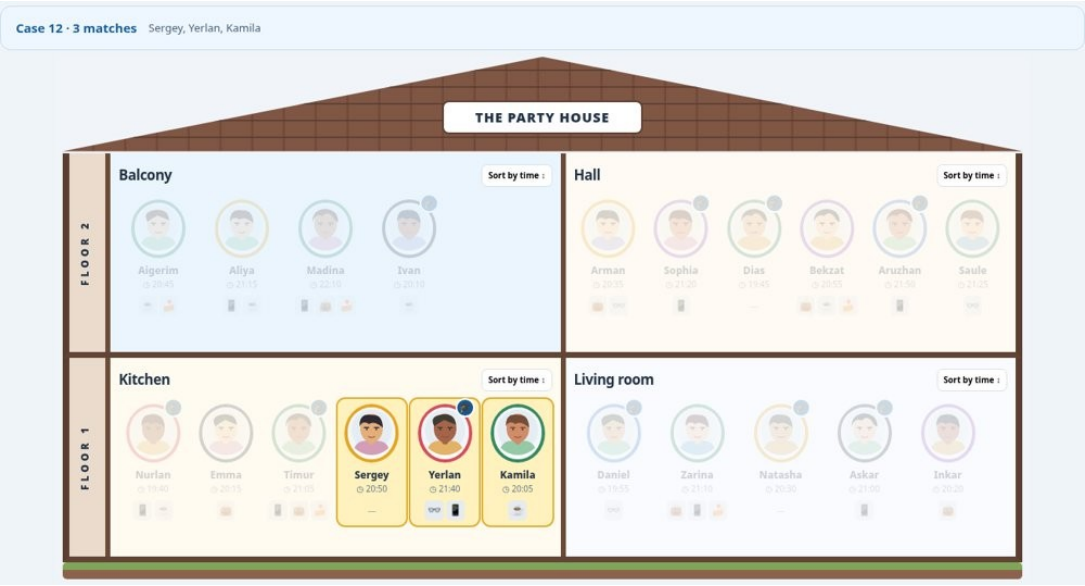



### Case 12 — After Timur?

$(`.guest[data-name="Timur"]`).nextAll(".guest")

**Expected output:** Sergey, Yerlan, Kamila

Explanation: The jQuery chain filters the page elements using DOM attributes, classes, position or related guests; it does not search the source array.

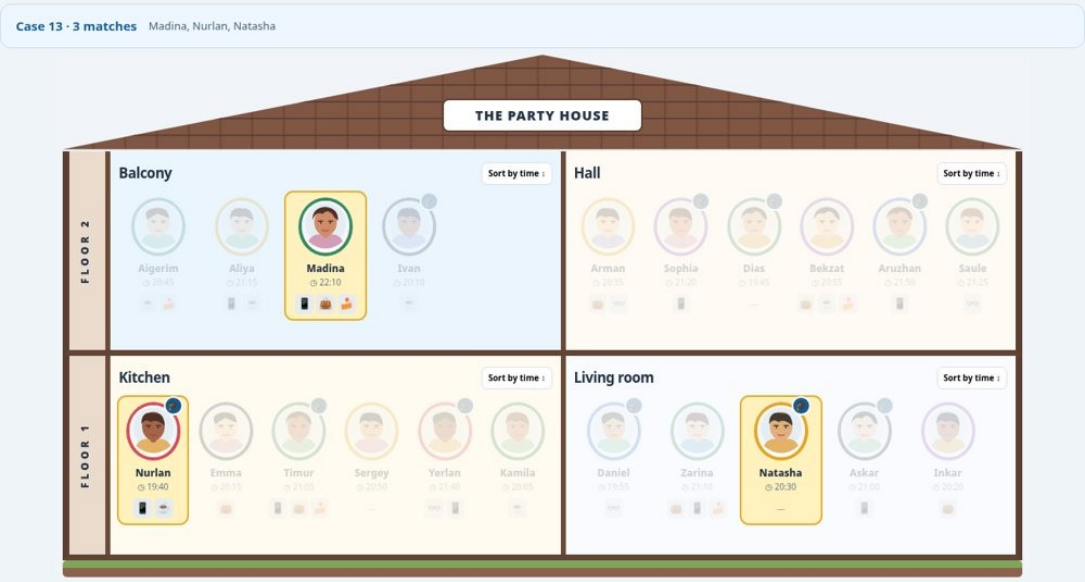



### Case 13 — Before someone in black?

$(`.guest.color-black`).prev(".guest")

**Expected output:** Madina, Nurlan, Natasha

Explanation: The jQuery chain filters the page elements using DOM attributes, classes, position or related guests; it does not search the source array.

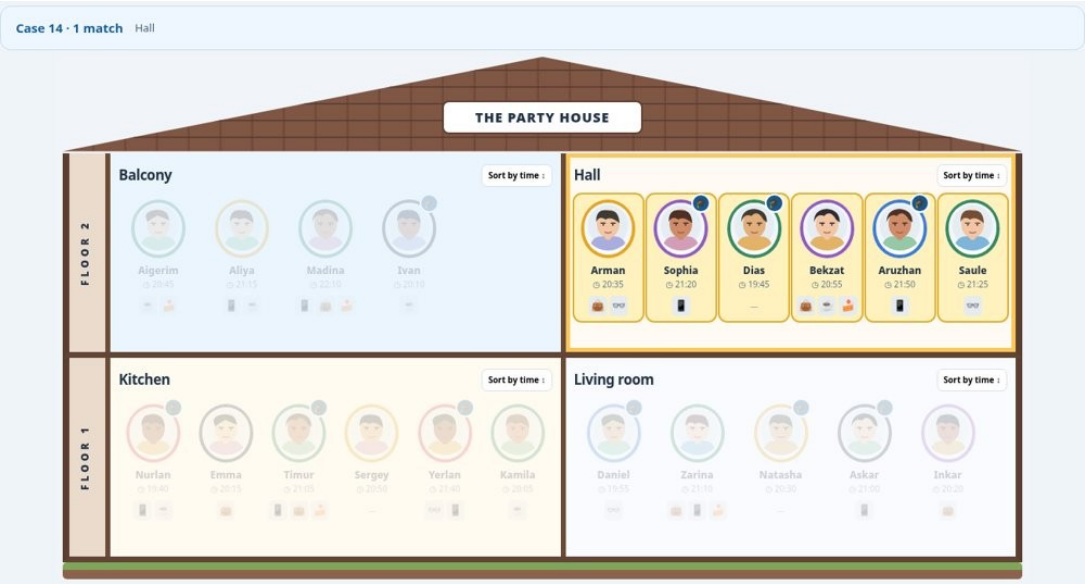



### Case 14 — Room of Aruzhan?

$(`.guest[data-name="Aruzhan"]`).closest(".room")

**Expected output:** Hall

Explanation: The jQuery chain filters the page elements using DOM attributes, classes, position or related guests; it does not search the source array.




### Case 15 — Rooms with fewer than 6?

$(`.room`).filter(function(){return $(this).find(".guest").length<6;})

**Expected output:** Balcony, Living room

Explanation: The jQuery chain filters the page elements using DOM attributes, classes, position or related guests; it does not search the source array.




### Case 16 — Cup but no glasses?

```
$(`.room:has(.guest.has-cup)`).filter(function(){return $(this).find(".guest.has-glasses").length===0;})
```

**Expected output:** Balcony

Explanation: The jQuery chain filters the page elements using DOM attributes, classes, position or related guests; it does not search the source array.

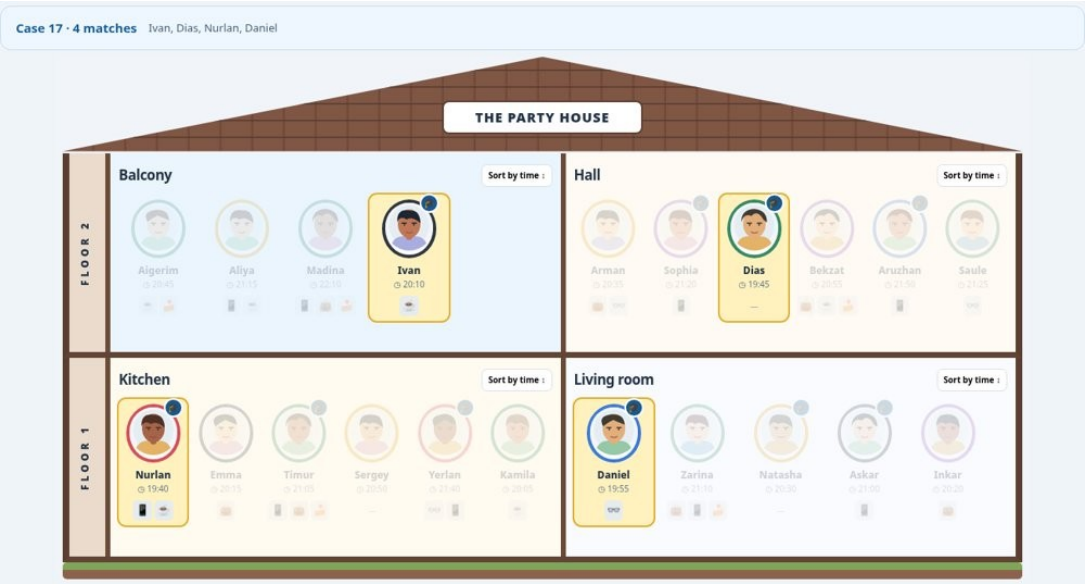



### Case 17 — First arrival to each room?

```
$(`.room .guest`).filter(function(){return +$(this).attr("data-minutes")===Math.min(...$(this).closest(".room").find(".guest").map(function(){return +$(this).attr("data-minutes");}).get());})
```

**Expected output:** Ivan, Dias, Nurlan, Daniel

Explanation: The jQuery chain filters the page elements using DOM attributes, classes, position or related guests; it does not search the source array.

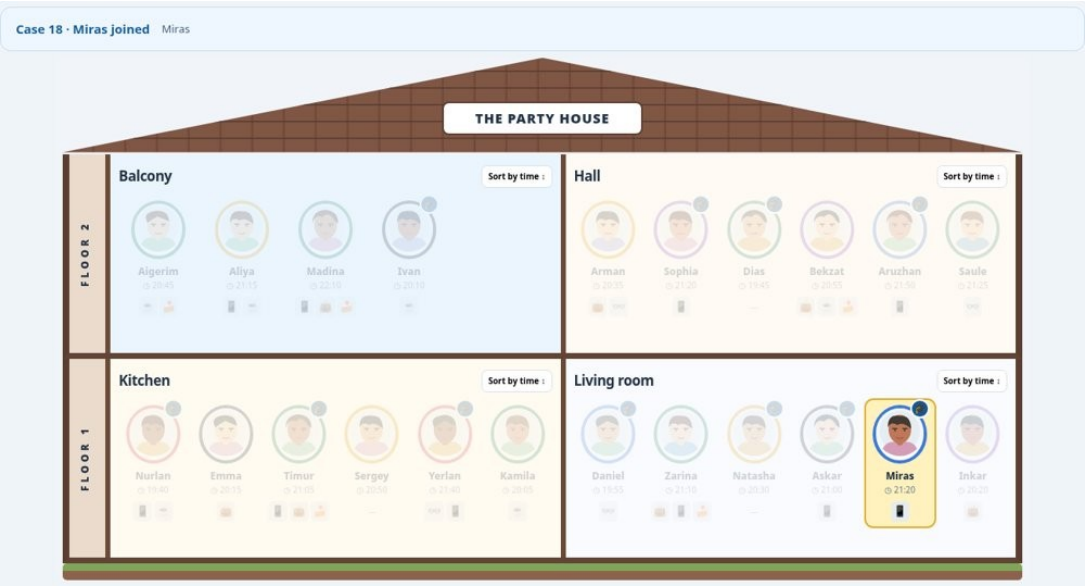



### Case 18 — Miras joins, fades in?

$(`.guest[data-name="Miras"]`)

**Expected output:** Miras appears between Askar and Inkar, 21:20

Action: Create a blue AITU student with a phone and arrival 21:20, insertAfter Askar, and show with fadeIn().

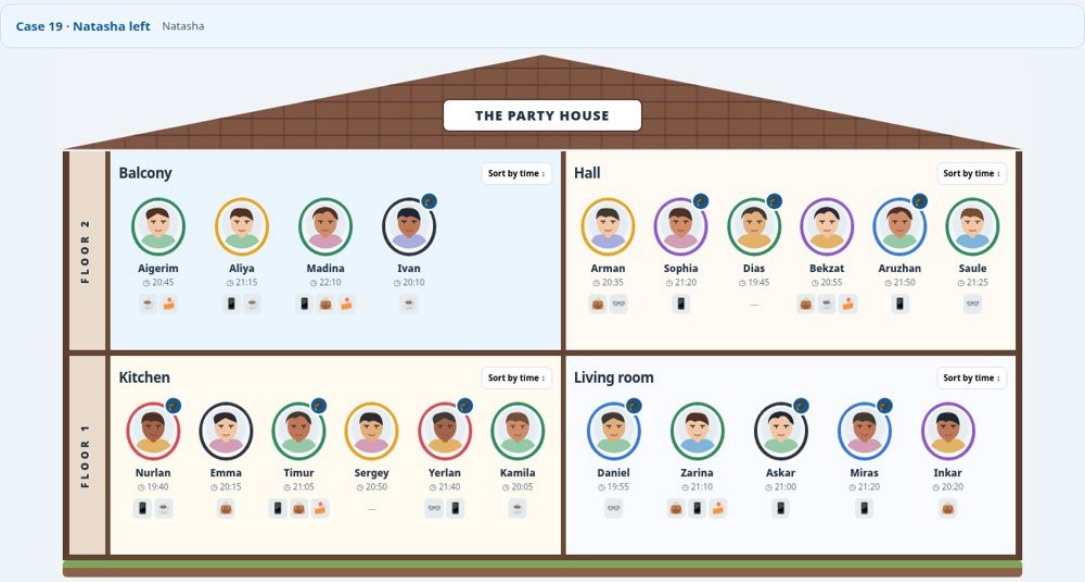



### Case 19 — Natasha leaves, fades out?

$(`.guest[data-name="Natasha"]`)

**Expected output:** Natasha fades out and is removed

Action: fadeOut(), remove() the guest card and update the underlying array.

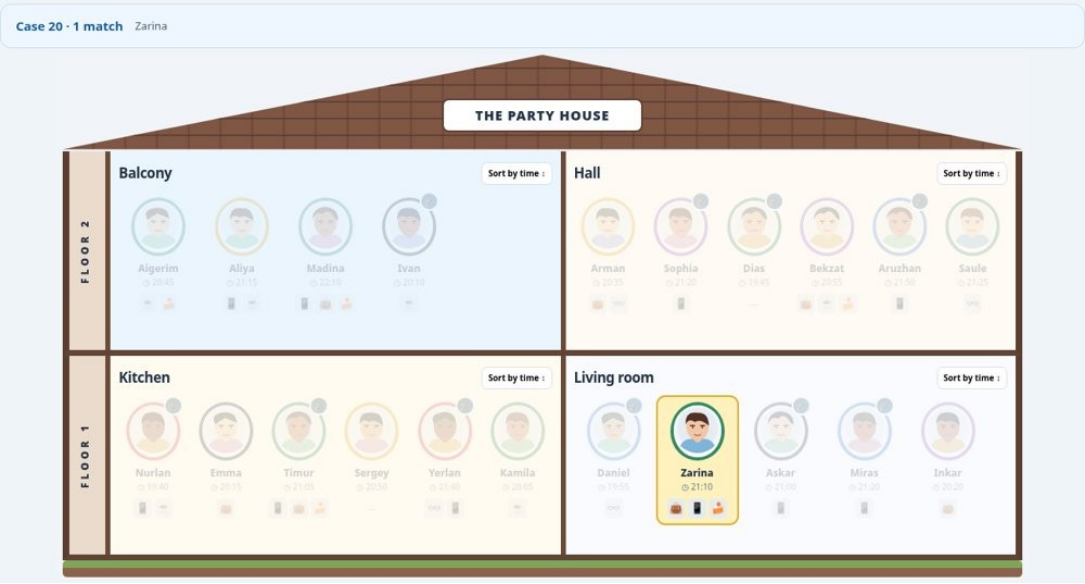



### Case 20 — Who stole the cake?

```
$(`.floor[data-floor="1"] .guest.has-crumbs:not(.has-cup)`).filter(function(){let t=+$(this).attr("data-minutes");return t>1260&&t<1290&&($(this).prev(".guest").hasClass("aitu")||$(this).next(".guest").hasClass("aitu"));})
```

**Expected output:** Zarina

Reasoning: Zarina is downstairs, arrived at 21:10, has cake crumbs and no cup; Daniel next to her is an AITU student.




## 4. Task 2 — jQuery effects and animations

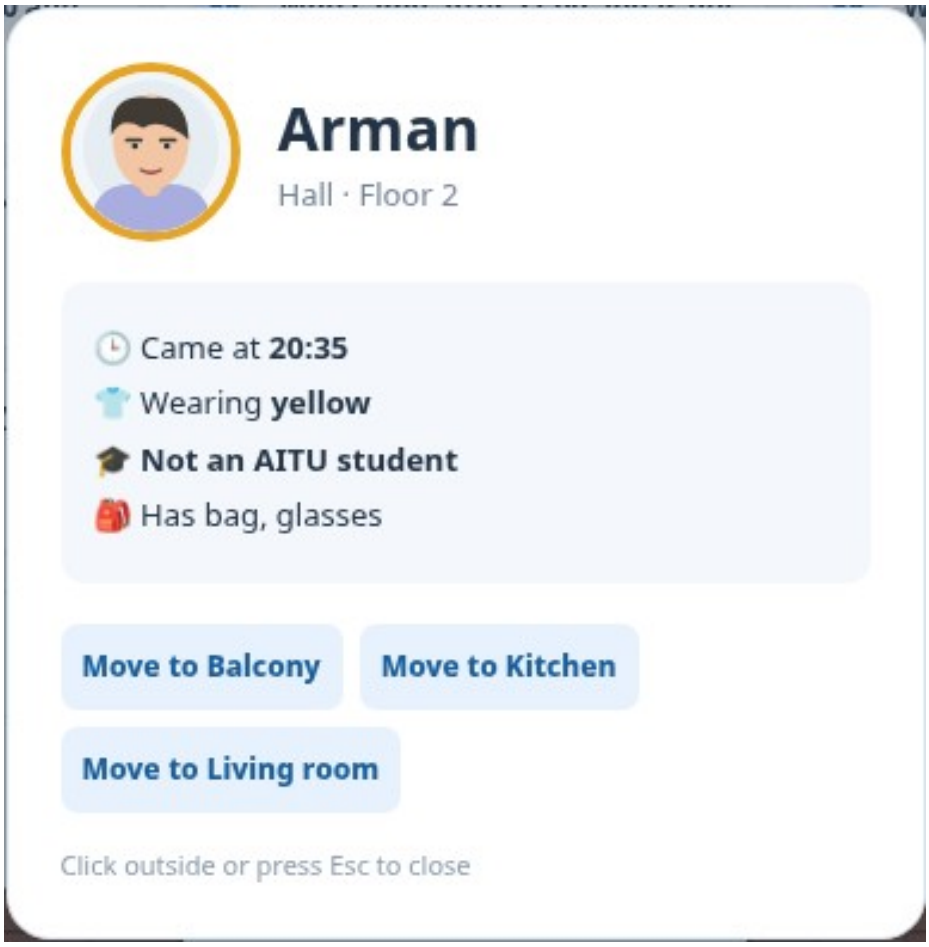
### Spotlight and Clear

fadeTo(300, 0.2) dims other guests while matching cards remain bright; Clear returns all to opacity 1.



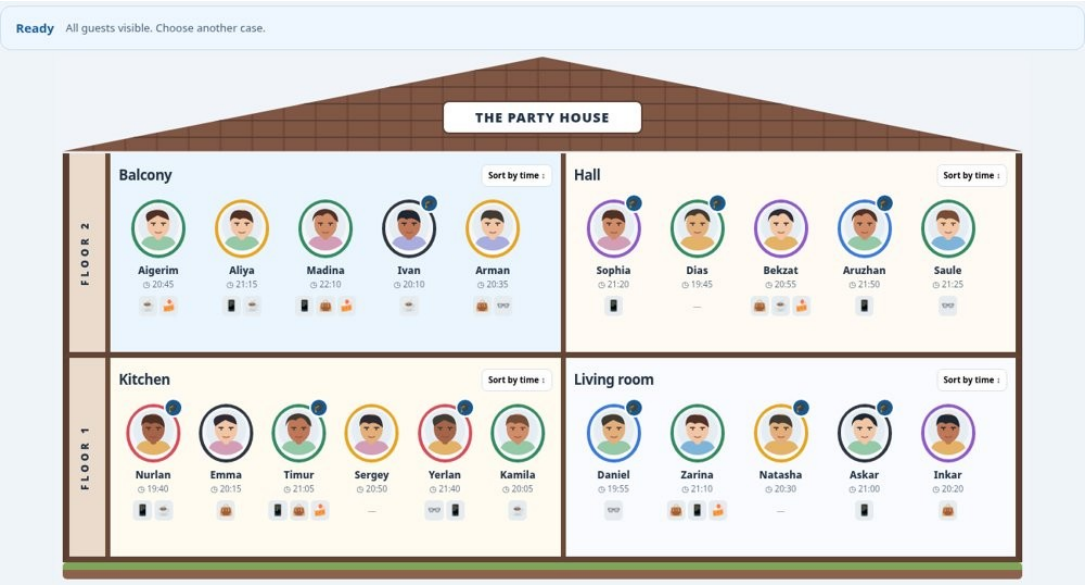

### Open and close details

animate() grows the guest details from the avatar; clicking outside or Escape animates it back.



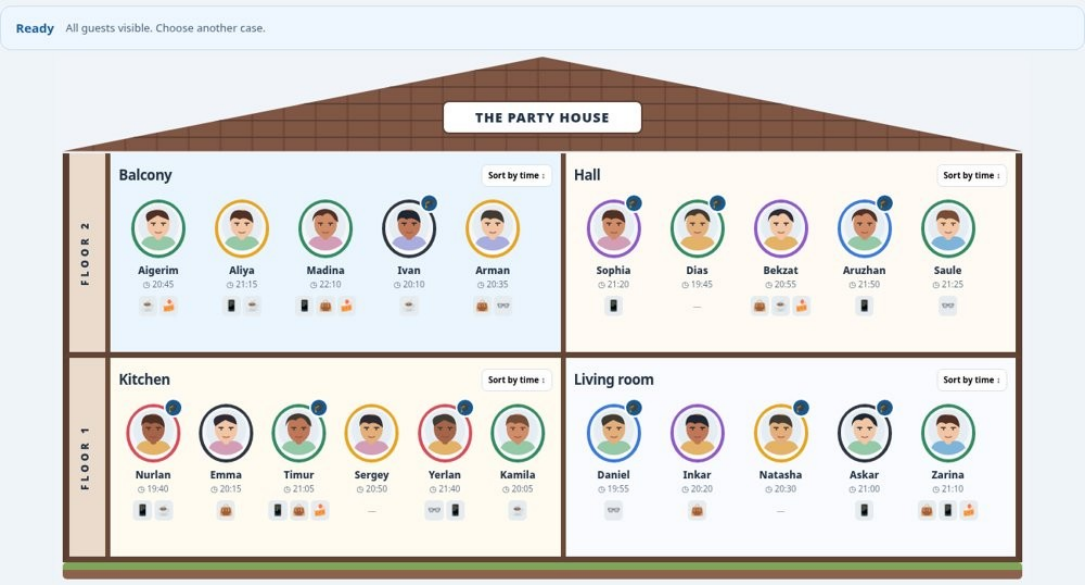

### Move between rooms

The guest is cloned for a flying animation; the source gap closes, the destination gap opens, and the model is updated.




### Sort by time

The room toggles between ascending arrival times and original illustration order; jQuery animates clones to their new positions.




### Fast repeated clicking

stop(true,true) clears pending animations; sort clones are stopped/removed before starting the next sort.




Required check: living room sorted = Daniel (19:55), Inkar (20:20), Natasha (20:30), Askar (21:00), Zarina (21:10). Moving Arman to Balcony removes him from the Hall case result.

## 5. Conclusion

During this assignment I practiced building elements from structured data with jQuery, selecting and traversing the DOM, and making interactions with fadeIn(), fadeOut(), fadeTo(), animate(), stop(), cloning and moving cards. I also learned why the DOM and the underlying array must remain synchronized. [Rewrite this short conclusion in your own words.]

## 6. Links and submission checklist

- GitHub repository URL: [PASTE THE ACTUAL URL]
- Task 1 source files: Task1/index.html, Task1/app.js, Task1/styles.css, Task1/jquery.min.js
- Task 2 source files: Task2/index.html, Task2/app.js, Task2/styles.css, Task2/jquery.min.js
- Review provided browser screenshots, and add your own where appropriate
- Check your name/group and explain the code at the defense